

SWE 4638

CSS Fundamentals

Styling & Layout for the Web

What is CSS?

Cascading Style Sheets

CSS controls how HTML elements look — colors, fonts, spacing, layout, and responsive behavior. It separates content (HTML) from presentation (CSS).

External

```
<link rel="stylesheet"
href="style.css">
```

Separate .css file (recommended)

Internal

```
<style>
  h1 { color: blue; }
</style>
```

Inside <head> tag

Inline

```
<p style="color:red;">
  Text here
</p>
```

On the element (avoid)

CSS Syntax: Rules & Declarations

h1

Selector

{

```
color: blue;  
font-size: 24px;  
margin-top: 10px;
```

}

Property : Value (Declaration)

Selector

Targets which HTML elements to style

Property

Which aspect to change (color, font, margin...)

Value

What to set it to (blue, 24px, bold...)

CSS Selectors

Element

```
h1 { }
```

Selects all <h1> elements

Class

```
.intro { }
```

Selects elements with class="intro"

ID

```
#header { }
```

Selects the element with id="header"

Universal

```
* { }
```

Selects every element on the page

Descendant

```
div p { }
```

Selects <p> inside <div>

Child

```
div > p { }
```

Selects direct <p> children of <div>

Attribute Selectors

[attr]

Elements with the attribute

```
input[required] { border: 2px solid red; }
```

[attr=val]

Exact match

```
input[type="text"] { padding: 8px; }
```

[attr^=val]

Starts with val

```
a[href^="https"] { color: green; }
```

[attr\$=val]

Ends with val

```
a[href$=".pdf"] { font-weight: bold; }
```

[attr*=val]

Contains val

```
img[src*="thumb"] { width: 100px; }
```

[attr~=val]

Word in space-separated list

```
p[class~="highlight"] { background: yellow; }
```

Structural & Form Pseudo-classes

Structural Selectors

Selector	Description
<code>:first-of-type</code>	First of its type in parent
<code>:last-of-type</code>	Last of its type
<code>:nth-of-type(n)</code>	Nth element of its type
<code>:only-child</code>	Only child in parent
<code>:empty</code>	Element with no children
<code>::before</code> / <code>::after</code>	Insert generated content

Form Pseudo-classes

Selector	Description
<code>:checked</code>	Checked radio/checkbox
<code>:disabled</code>	Disabled form element
<code>:enabled</code>	Enabled form element
<code>:required</code>	Required field
<code>:valid</code> / <code>:invalid</code>	Validation state
<code>::placeholder</code>	Placeholder text style

```
/* Style all required inputs with valid data */  
input:required:valid { border: 2px solid green; }
```

```
/* Add icon after external links */  
a[href^="http"]::after { content: " \2197"; }
```

Pseudo-classes & Combinators

Pseudo-classes

Selector	When Active
<code>:hover</code>	Mouse over element
<code>:active</code>	While being clicked
<code>:focus</code>	When focused (tab/click)
<code>:first-child</code>	First child of parent
<code>:nth-child(n)</code>	Nth child of parent
<code>:not(.cls)</code>	Elements without .cls

Combining Selectors

Syntax	Meaning
<code>A, B</code>	Both A and B (union)
<code>A B</code>	B nested inside A
<code>A > B</code>	B direct child of A
<code>A + B</code>	B immediately after A
<code>A ~ B</code>	B anywhere after A
<code>A.cls</code>	A with class .cls

Class Exercise 1: CSS Selectors Challenge

Given this HTML, write CSS selectors to achieve the following:

```
<nav>
  <a href="https://google.com"
class="external">Google</a>
  <a href="/about" class="nav-link active">About</a>
  <a href="/contact" class="nav-link">Contact</a>
</nav>
<form>
  <input type="text" required placeholder="Name">
  <input type="email" placeholder="Email">
  <button disabled>Submit</button>
</form>
```

Your Tasks:

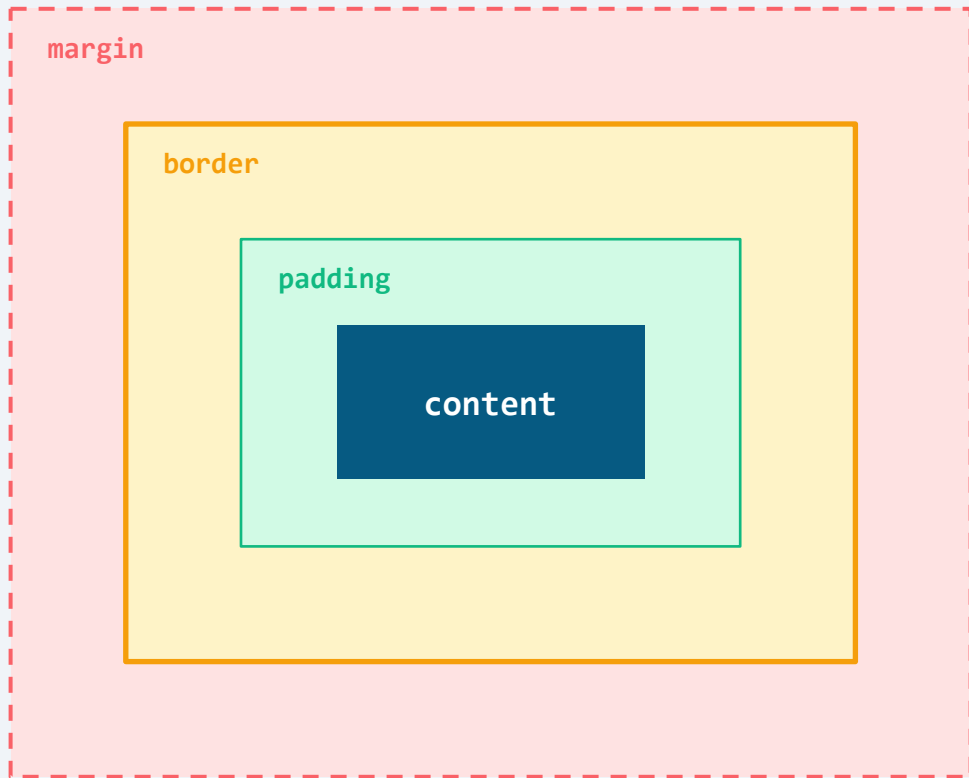
1. Style only links starting with https
2. Make the .active link bold and red
3. Style disabled buttons gray
4. Add a red border to required inputs
5. Style the first <a> inside <nav>

Try it: 5 minutes | Hint: Use attribute selectors and pseudo-classes!

The Box Model

Understanding How Every Element Takes Up Space

The CSS Box Model



Total Width =

content width
+ left/right padding
+ left/right border
+ left/right margin

box-sizing: border-box

Makes width/height include padding and border. Much easier to work with! Add to * { } in your CSS.

CSS Units

Absolute Units

px Pixels — most common fixed unit

pt Points (1pt = 1/72 inch)

cm / mm Centimeters / Millimeters

Rule of Thumb

Use rem for fonts, % or vw for widths, px only for borders and small fixed values. This makes your designs responsive.

Relative Units (Preferred)

em Relative to parent font-size

rem Relative to root font-size (preferred)

vw 1% of viewport width

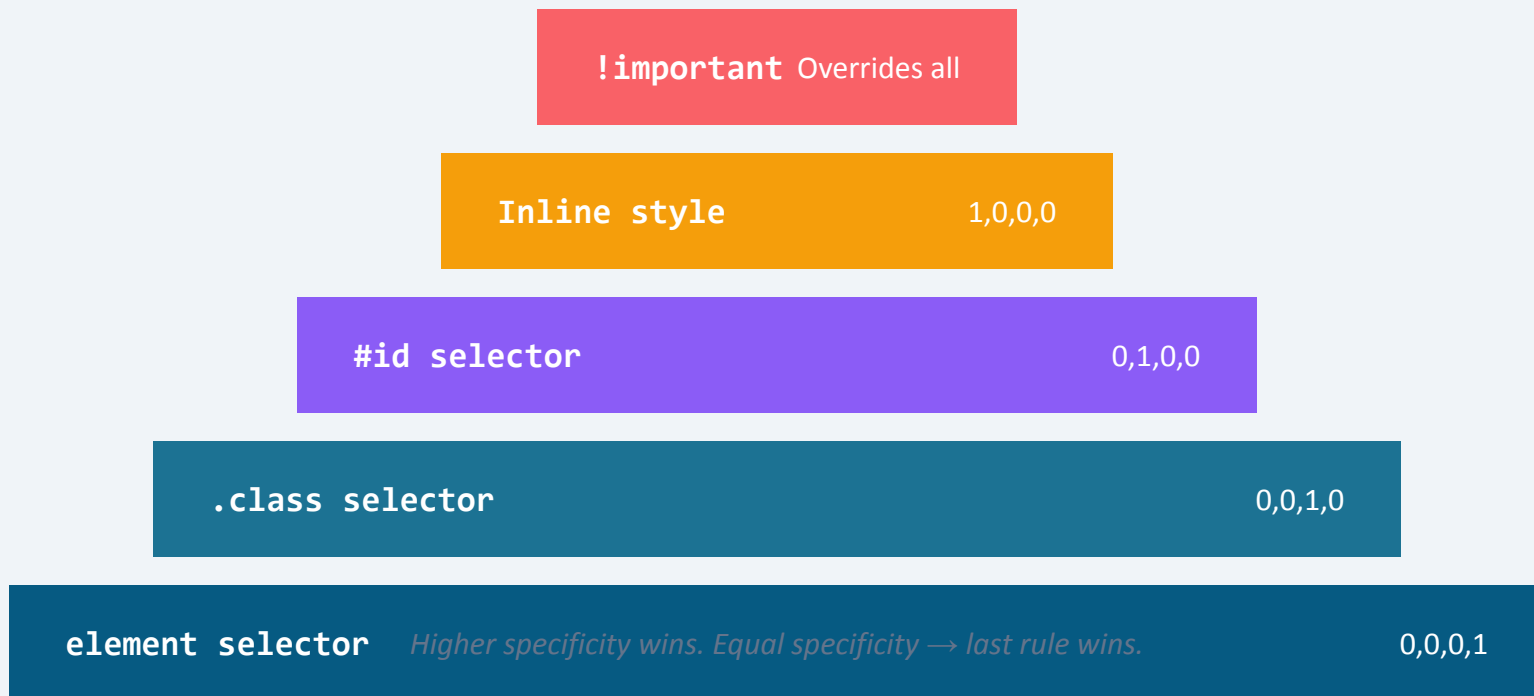
vh 1% of viewport height

% Relative to parent element

fr Fraction of available space (Grid)

Cascading & Specificity

When multiple rules target the same element, CSS uses specificity to decide which wins.



Display & Positioning

display Property

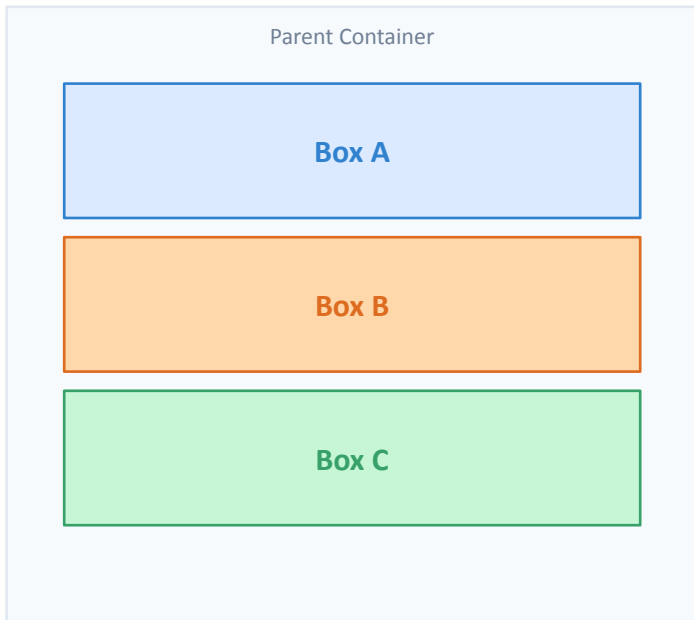
block	Full width, starts on new line (div, p, h1)
inline	Only as wide as content (span, a, strong)
inline-block	Inline but respects width/height
none	Hidden — removed from layout
flex	Enables Flexbox layout
grid	Enables Grid layout

position Property

static	Default — normal document flow
relative	Offset from normal position
absolute	Positioned relative to nearest positioned ancestor
fixed	Stays in place during scroll
sticky	Normal until scroll threshold, then fixed

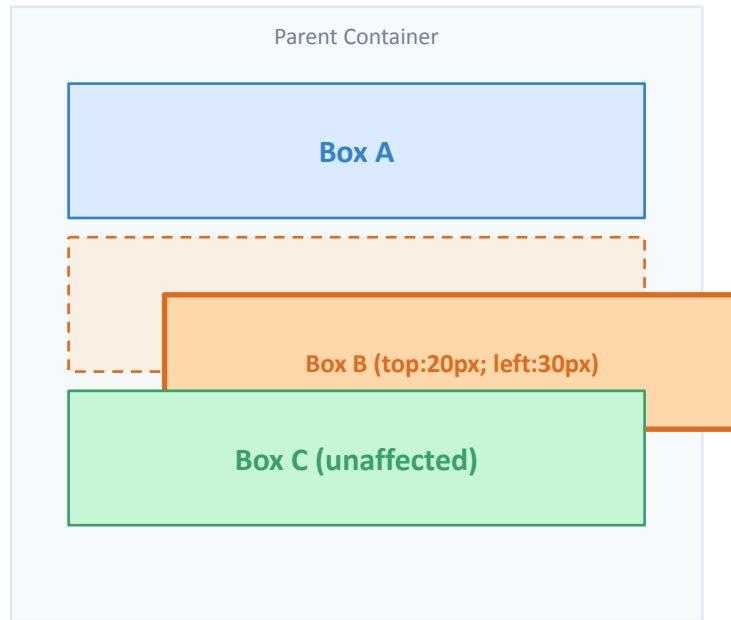
Positioning: static vs relative

position: static (default)



Elements stack in normal document flow

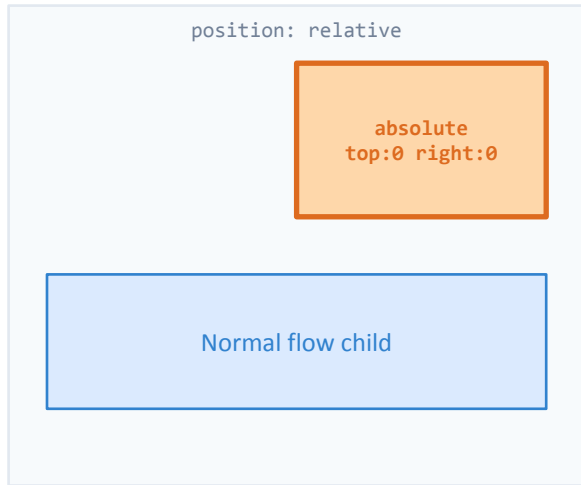
position: relative



Moves from its original position; other elements stay put

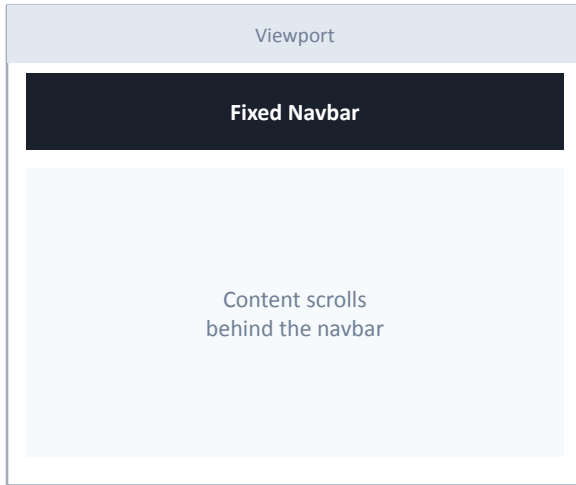
Positioning: absolute vs fixed vs sticky

position: absolute



Removed from flow.
Positioned to ancestor.

position: fixed



Stays in place relative
to viewport on scroll.

position: sticky



Normal flow until scroll
threshold, then sticks.

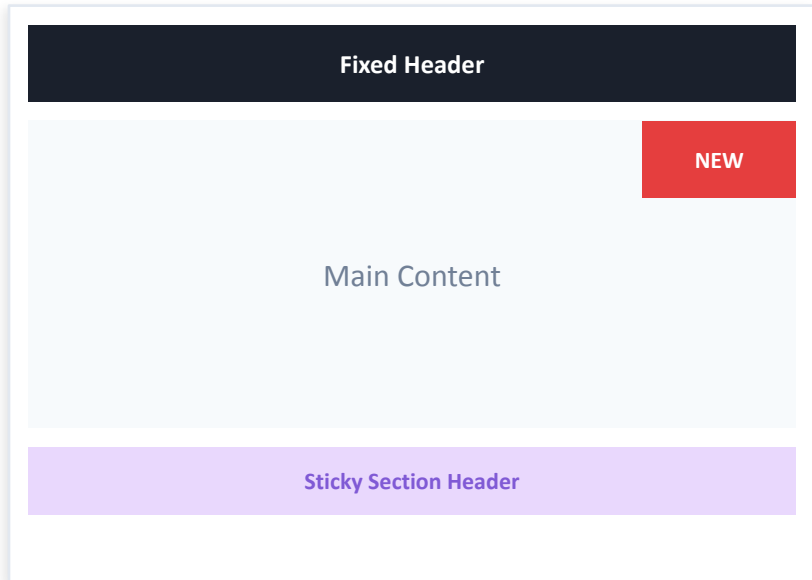
```
.badge { position: absolute; top: 0; right: 0; }  
position: sticky; top: 0; }
```

```
.navbar { position: fixed; top: 0; width: 100%; }
```

```
.header {
```

Class Exercise 2: CSS Positioning

Recreate this layout using CSS positioning:



Requirements:

1. Create a fixed header using `position: fixed`
2. Add a "NEW" badge that stays in the top-right corner of a card (use `absolute`)
3. Make a section header sticky with `top: 0`
4. Ensure content scrolls under the fixed header (add `padding-top`!)

Try it: 8 minutes | Use the browser DevTools to test your positioning!

CSS Flexbox

One-Dimensional Layout Made Simple

Flexbox: Container Properties

display: flex turns a container into a flex container. Its direct children become flex items.

justify-content (main axis)

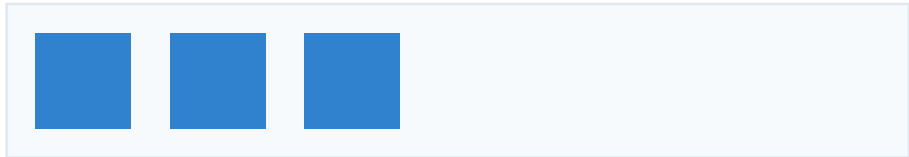
- flex-start — items at start (default)
- flex-end — items at end
- center — items centered
- space-between — equal space between items
- space-around — equal space around items
- space-evenly — even spacing everywhere

align-items (cross axis)

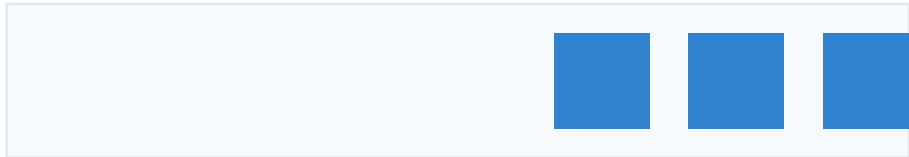
- stretch — fill container height (default)
- flex-start — items at top
- flex-end — items at bottom
- center — items vertically centered
- baseline — align by text baseline

Flexbox Visual: justify-content

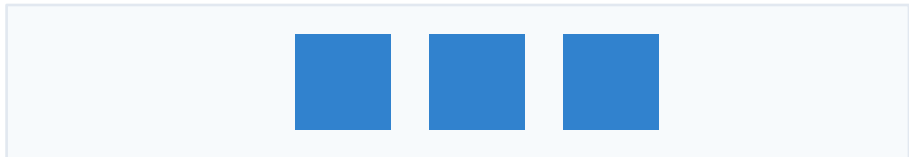
flex-start



flex-end



center



space-between



space-around



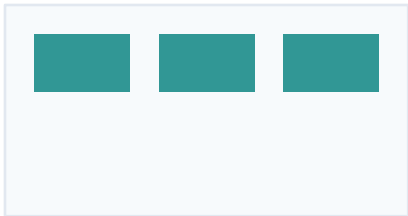
space-evenly



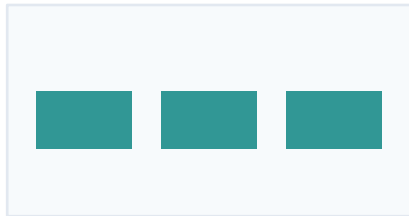
Flexbox Visual: align-items & direction

align-items

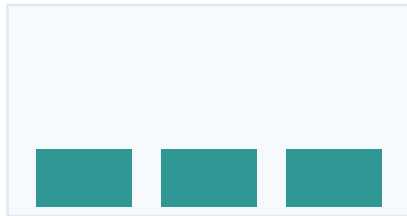
flex-start



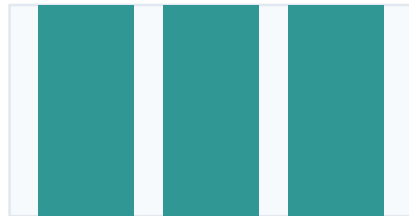
center



flex-end

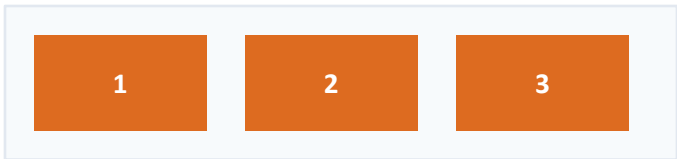


stretch

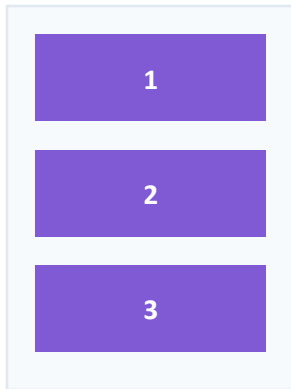


flex-direction

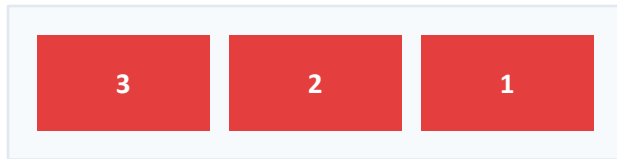
row



column



row-reverse



Flexbox: Item Properties & Direction

flex-direction & flex-wrap

```
flex-direction: row;  
flex-direction: column;  
flex-direction: row-reverse;  
flex-wrap: wrap;
```

Item Properties

```
flex: 1;           /* grow to fill */  
order: 2;          /* reorder */  
align-self: end;   /* override */
```

Common Pattern: Centering Anything

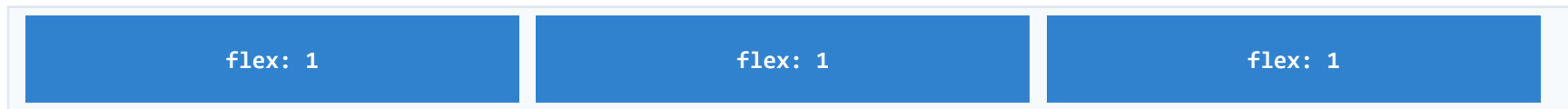
```
.center-me {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

This is the most common flexbox pattern you'll use. It perfectly centers any child element both horizontally and vertically.

Flexbox Visual: flex-grow & flex-shrink

flex-grow: How items share available space

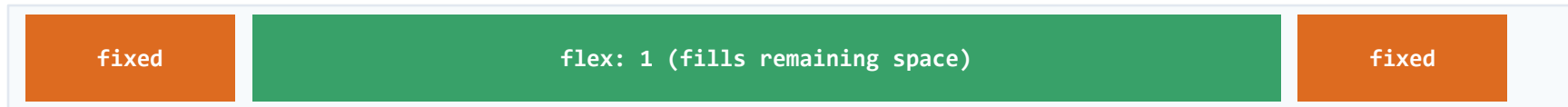
All flex-grow: 1 (equal share)



flex-grow: 1, 2, 1 (middle gets double)



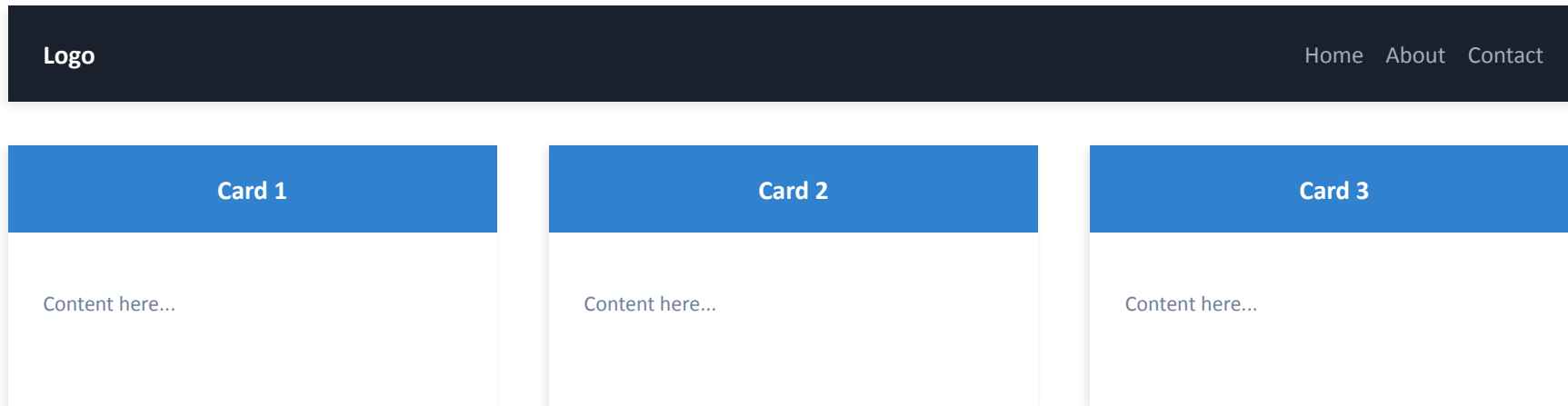
flex-grow: 0, 1, 0 (only middle grows)



Common layout: sidebar (fixed width) + main content (flex: 1). This is used in dashboards, admin panels, and email clients.

Class Exercise 3: Build a Flexbox Layout

Create this navigation bar and card layout using Flexbox:



Requirements:

1. Nav: display: flex, justify-content: space-between, align-items: center
2. Cards: wrap 3 cards in a flex container with gap: 20px
3. Each card: flex: 1 so they share space equally
4. Make it responsive: flex-direction: column on small screens

Responsive Design & Media Queries

Responsive design means your website looks good on all screen sizes — from phones to wide monitors.

Media Queries

```
/* Mobile first approach */
.container {
  flex-direction: column;
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}
```

Common Breakpoints

480px Small phones

768px Tablets

1024px Laptops

1280px Desktops

1536px Large screens

Essential CSS Properties

Typography

font-family, font-size,
font-weight, line-height,
text-align,
text-decoration,
letter-spacing

Colors & BG

color, background-color,
background-image, opacity,
gradient

Spacing

margin, padding, gap
(flex/grid), width, height,
max-width, min-height

Borders

border, border-radius,
outline, box-shadow

Layout

display, position,
top/right/bottom/left,
z-index, overflow, float

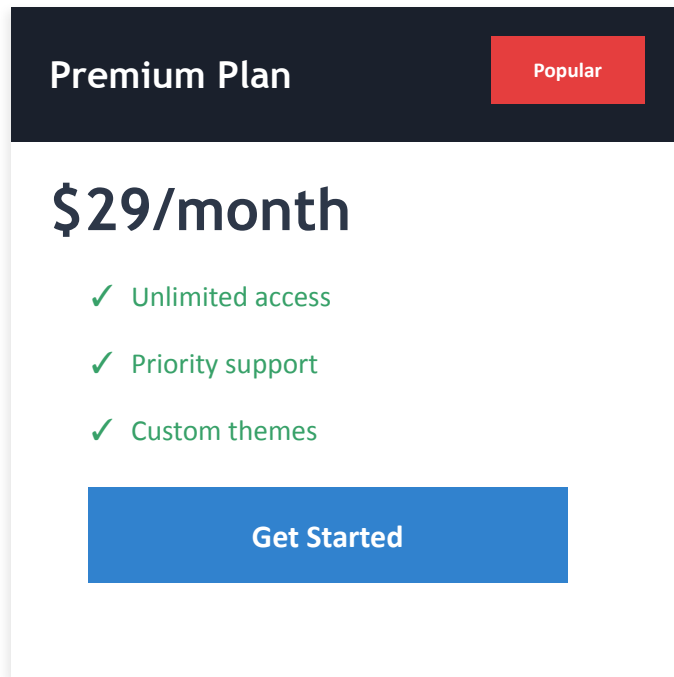
Transitions

transition, transform,
animation, @keyframes

Class Exercise 4: Style a Complete Card Component

Using everything learned today, style this card component:

```
<div class="card">
  <div class="card-header">
    <h2>Premium Plan</h2>
    <span class="badge">Popular</span>
  </div>
  <div class="card-body">
    <p class="price">$29/month</p>
    <ul>
      <li>Unlimited access</li>
      <li>Priority support</li>
      <li>Custom themes</li>
    </ul>
    <button class="btn">Get Started</button>
  </div>
</div>
```



Apply: box model, flexbox, colors, borders, hover states, and units!

Summary

- CSS separates presentation from structure — always use external stylesheets
- Selectors target elements by tag, class, ID, or relationships
- The box model (content + padding + border + margin) controls spacing
- Flexbox simplifies one-dimensional layouts and centering
- Media queries enable responsive designs for all screen sizes
- Use rem units and mobile-first approach for modern CSS

Next Lecture: JavaScript & DOM Manipulation